TechRise 3.0

Cybersecurity & DevSecOps · Abia Cohort 3.0 · 2026

WEEK 7 · DAY 1 · SNORT 3.0

IDS Installation, Rule Writing and Testing Guide

This guide is written specifically for Snort 3.0

Student Name: _____

SNORT 2.x vs SNORT 3.0 — WHAT CHANGED

Snort 3.0 is a complete rewrite of Snort 2.x. The rule syntax is mostly the same but the configuration system, command-line flags, and some detection keywords are different. This guide uses Snort 3.0 throughout.

	Snort 2.x	Snort 3.0 (what you have)
Config file	snort.conf (text)	snort.lua (Lua-based)
Validation	snort -T -c snort.conf	snort -c snort.lua --warn-all
Running	snort -A console -c snort.conf -i eth0	snort -c snort.lua -i eth0 -A alert_fast
Alert output flag	-A console	-A alert_fast or -A alert_full
Alert log file	/var/log/snort/alert	/var/log/snort/alert_fast.txt
http_uri	keyword modifier	sticky buffer (different syntax)
threshold keyword	threshold:type threshold,...	detection_filter:track by_src,...
Rule files	include in snort.conf	ips section in snort.lua

What stays the same in Snort 3.0

- Basic rule structure: action protocol src_ip src_port -> dst_ip dst_port (options)
- content: keyword for payload matching
- nocase modifier
- flow: keyword
- dsize: keyword for packet size matching
- sid: and rev: rule identifiers
- All five PayliteNG rules work in Snort 3.0 with small syntax updates

PART 1 — INSTALL AND CONFIGURE SNORT 3.0

STEP 1 Find Your Network Interface

BASH

```
ip a | grep -E '^[0-9]+:' | awk '{print $2}' | grep -v lo # Write your interface name here:
_____________ # You need this for all Snort commands today
```

STEP 2 Install Snort 3.0

BASH

```
sudo apt update sudo apt install snort -y # Verify version — must show 3.x snort --version #
Expected output: # ,,_ -*> Snort! <*- # o" )~ Version 3.x.x # '''' By Martin Roesch & The Snort
Team
```

✓ Snort 3.0 confirmed

```
Version 3.x.x GRE (Build xxxx) ← Confirms Snort 3.0 is installed
```

STEP 3 Locate the Snort 3.0 Config File

Snort 3.0 uses a Lua-based configuration file — not the old text-based snort.conf from version 2.x.

BASH

```
# Find the config file location ls /etc/snort/ # Snort 3 uses snort.lua (NOT snort.conf) ls -la
/etc/snort/snort.lua # Also check for rules directory ls /etc/snort/rules/ 2>/dev/null || ls
/usr/share/snort/rules/ 2>/dev/null
```

✓ Snort 3.0 config found

```
/etc/snort/ snort.lua ← Snort 3.0 config file (Lua-based) rules/ ← rules directory ← snort.lua is the
file — NOT snort.conf
```

■ If you only see snort.conf but not snort.lua

You may have Snort 2.x installed. Run: snort --version to confirm. This guide is for Snort 3.0 only.

STEP 4 Create the Local Rules File

BASH

```
# Create the rules directory if it does not exist sudo mkdir -p /etc/snort/rules # Create your
local rules file sudo touch /etc/snort/rules/local.rules # Verify it exists ls -la
/etc/snort/rules/local.rules
```

STEP 5 Tell Snort 3.0 to Load Your Local Rules

In Snort 3.0 you need to include your local.rules file in snort.lua:

BASH + EDITOR

```
sudo nano /etc/snort/snort.lua # Find the ips section — it looks like this: # ips = # { # -- use this to enable default rules # -- enable_builtin_rules = true, # # -- list of rules files # -- include = 'snort3-community.rules', # } # Add your local rules file inside the ips section: # ips = # { # include = '/etc/snort/rules/local.rules', # } # Save: Ctrl+O then Enter, exit: Ctrl+X
```

STEP 6 Validate the Snort 3.0 Configuration

Snort 3.0 validation command is different from 2.x:

Snort 2.x (old)	Snort 3.0 (use this)
`# Snort 2.x (OLD — do not use) sudo snort -T -c /etc/snort/snort.conf`	`# Snort 3.0 (CORRECT) sudo snort -c /etc/snort/snort.lua --warn-all`

✓ Config valid in Snort 3.0

```
Snort successfully loaded all rules and default settings! ← This is the Snort 3.0 success message ← Different wording from Snort 2.x
```

✓ Part 1 Complete — Snort 3.0 Ready

- ■ snort --version shows Version 3.x
- ■ /etc/snort/snort.lua exists
- ■ /etc/snort/rules/local.rules created
- ■ local.rules path added to ips section in snort.lua
- ■ sudo snort -c /etc/snort/snort.lua --warn-all passes

PART 2 — SNORT 3.0 RULE ANATOMY

The basic rule structure is the same as Snort 2.x. Most differences are in detection options.

SNORT 3.0 RULE FORMAT

```
alert tcp any any -> any 80 (msg:"HTTP traffic"; sid:1000001; rev:1;)
```

Field	Example	What It Means
action	alert	What to do: alert, drop, pass, block, react
protocol	tcp	tcp, udp, icmp, ip
source IP	any	Source address — any = all, $HOME_NET = your network
source port	any	Source port — any = all
direction	->	-> one way, <> bidirectional
dest IP	any	Destination — $HTTP_SERVERS = web servers
dest port	80	Destination port — $HTTP_PORTS = web ports
msg	"HTTP traffic"	Alert message — make it descriptive
sid	1000001	Unique rule ID — use 1000000+ for local rules
rev	1	Revision number — increment when you update

Key Syntax Changes in Snort 3.0

Three things changed from Snort 2.x that you must know:

Change 1 — threshold is now detection_filter:

Snort 2.x (old)	Snort 3.0 (use this)
`# Snort 2.x threshold:type threshold, track by_src, count 5, seconds 60;`	`# Snort 3.0 detection_filter:track by_src, count 5, seconds 60;`

Change 2 — http_uri is a sticky buffer (options use comma not semicolon):

Snort 2.x (old)	Snort 3.0 (use this)
`# Snort 2.x http_uri; content:"UNION"; nocase; content:"SELECT"; nocase; distance:0;`	`# Snort 3.0 http_uri; content:"UNION", nocase; content:"SELECT", nocase, distance:0;`

Change 3 — Running Snort and alert output:

Snort 2.x (old)	Snort 3.0 (use this)
`# Snort 2.x sudo snort -A console \ -c /etc/snort/snort.conf \ -i eth0`	`# Snort 3.0 sudo snort \ -c /etc/snort/snort.lua \ -i eth0 \ -A alert_fast`

PART 3 — WRITE THE PAYLITENG DETECTION RULES (SNORT 3.0)

■ PayliteNG Context

Every rule maps to the PayliteNG attack chain. @gr4yh4t performed five attack types. You will write one Snort 3.0 rule for each — the rules that would have caught the breach on January 13th.

STEP 1 Open the Local Rules File

BASH

```
sudo nano /etc/snort/rules/local.rules # Write all five rules here # Save: Ctrl+O Enter Exit: Ctrl+X
```

RULE 1 ICMP Detection

Detects: Any ping or ICMP traffic — connection test

Purpose: Verify Snort 3.0 is detecting traffic before writing security rules.

ADD TO local.rules — Rule 1

```
alert icmp any any -> any any (msg:"TECHRISE ICMP Detected"; sid:1000001; rev:1;)
```

- Identical in Snort 2.x and 3.0 — basic rule syntax unchanged
- sid:1000001 — local rules always use 1000000+

RULE 2 SSH Brute Force Detection

Detects: Multiple failed SSH attempts — detection_filter in Snort 3.0

■ PayliteNG Context

@gr4yh4t made 47 SSH attempts in under 20 seconds. This rule uses detection_filter (Snort 3.0 replacement for threshold) to fire after 5 attempts from the same IP in 60 seconds.

Snort 3.0 version — uses detection_filter instead of threshold:

ADD TO local.rules — Rule 2 (Snort 3.0)

```
alert tcp any any -> $HOME_NET 22 (msg:"PAYLITENG SSH Brute Force Detected"; flow:to_server; detection_filter:track by_src, count 5, seconds 60; sid:1000002; rev:1;)
```

Snort 2.x (old)	Snort 3.0 (use this)
`# Snort 2.x (do not use) threshold:type threshold, track by_src, count 5, seconds 60;`	`# Snort 3.0 (correct) detection_filter:track by_src, count 5, seconds 60;`

- detection_filter — Snort 3.0 name for what was 'threshold' in 2.x
- track by_src — count per source IP
- count 5, seconds 60 — fire after 5 attempts in 60 seconds

RULE 3 — Anonymous FTP Login

Detects: FTP USER anonymous in packet payload

■ PayliteNG Context

@gr4yh4t used anonymous FTP to access PayliteNG's file server and download 45MB of customer data. This rule detects the 'USER anonymous' string in FTP traffic — identical syntax in Snort 3.0.

ADD TO local.rules — Rule 3 (Snort 3.0)

```
alert tcp any any -> $HOME_NET 21 (msg:"PAYLITENG Anonymous FTP Login Attempt";
flow:to_server,established; content:"USER anonymous"; nocase; sid:1000003; rev:1;)
```

- content: and nocase — identical syntax in Snort 3.0 and 2.x
- flow:established — only on fully established TCP connections
- No detection_filter needed — any anonymous FTP login is immediately suspicious

RULE 4 — SQL Injection Detection

Detects: UNION SELECT in HTTP URI — sticky buffer in Snort 3.0

■ PayliteNG Context

IP 203.0.113.88 sent SQL injection to PayliteNG's portal: GET /login.php?id=1+UNION+SELECT+user,password. In Snort 3.0 http_uri is a sticky buffer — content options use commas.

Snort 3.0 version — http_uri sticky buffer with comma-separated options:

ADD TO local.rules — Rule 4 (Snort 3.0)

```
alert tcp any any -> $HTTP_SERVERS $HTTP_PORTS (msg:"PAYLITENG SQL Injection UNION SELECT";
flow:to_server,established; http_uri; content:"UNION", nocase; content:"SELECT", nocase,
distance:0; sid:1000004; rev:1;)
```

Snort 2.x (old)	Snort 3.0 (use this)
`# Snort 2.x (semicolons) http_uri; content:"UNION"; nocase; content:"SELECT"; nocase; distance:0;`	`# Snort 3.0 (commas inside content) http_uri; content:"UNION", nocase; content:"SELECT", nocase, distance:0;`

- http_uri — sticky buffer in Snort 3.0: all following content options apply to the URI
- Options inside content() use commas in Snort 3.0 — not semicolons
- distance:0 — SELECT must immediately follow UNION in the URI

RULE 5 — Large FTP Transfer Detection

Detects: Outbound packets over 10KB — data exfiltration indicator

■ PayliteNG Context

@gr4yh4t downloaded customer_backup_2024.sql.gz — 45MB. Large outbound FTP packets are a data exfiltration indicator. dsize: syntax is identical in Snort 3.0.

ADD TO local.rules — Rule 5 (Snort 3.0)

```
alert tcp $HOME_NET 21 -> any any (msg:"PAYLITENG Large FTP Transfer — Possible Exfiltration";
flow:from_server; dsize:>10000; sid:1000005; rev:1;)
```

- dsize:>10000 — identical syntax in Snort 3.0
- flow:from_server — outbound traffic FROM the FTP server
- Source is $HOME_NET port 21 — traffic leaving YOUR server

✓ Verification Checklist — Tick When Done

- ■ All 5 rules written in /etc/snort/rules/local.rules
- ■ Rule 2 uses detection_filter (not threshold)
- ■ Rule 4 http_uri content options use commas (not semicolons)
- ■ File saved

PART 4 — VALIDATE RULES IN SNORT 3.0

Always validate after adding rules. Snort 3.0 validation command is different from 2.x.

Snort 2.x (old)	Snort 3.0 (use this)
`# Snort 2.x (do not use) sudo snort -T -c /etc/snort/snort.conf`	`# Snort 3.0 (use this) sudo snort -c /etc/snort/snort.lua --warn-all`

✓ Rules valid in Snort 3.0

```
Snort successfully loaded all rules and default settings! ← Snort 3.0 success message ← All 5 rules loaded without errors
```

■ Common Snort 3.0 errors

'Unknown rule option threshold' — you used Snort 2.x syntax. Replace with detection_filter. 'Content option after sticky buffer must use comma' — you used semicolons inside http_uri content options. Replace with commas.

✓ Verification Checklist — Tick When Done

- ■ sudo snort -c /etc/snort/snort.lua --warn-all passes with no errors

PART 5 — RUNNING SNORT 3.0

The run command is different in Snort 3.0. Alert output flags changed.

Running Snort 3.0 in NIDS Mode

Snort 2.x (old)	Snort 3.0 (use this)
`# Snort 2.x (do not use) sudo snort -A console \ -c /etc/snort/snort.conf \ -i eth0 \ -l /var/log/snort`	`# Snort 3.0 (use this) sudo snort \ -c /etc/snort/snort.lua \ -i eth0 \ -l /var/log/snort \ -A alert_fast`

Snort 3.0 Alert Output Options

- -A alert_fast — one line per alert (recommended for class demo)
- -A alert_full — full packet details per alert
- -A alert_json — JSON format (good for Splunk integration)
- -A alert_csv — CSV format
- Alerts saved to: /var/log/snort/alert_fast.txt

Full run command for class demo:

BASH — TERMINAL 1

```
# Replace eth0 with YOUR interface name # Keep Terminal 1 open — alerts appear here sudo snort \ -c /etc/snort/snort.lua \ -i eth0 \ -l /var/log/snort \ -A alert_fast # To also watch alerts as they fire: # Open a second terminal and run: sudo tail -f /var/log/snort/alert_fast.txt
```

✓ Snort 3.0 running

```
---------------------------------------------------- Commencing packet processing
---------------------------------------------------- ← Snort 3.0 is running and watching the network ←
Alerts appear in /var/log/snort/alert_fast.txt
```

■ In Snort 3.0 you may not see alerts directly in the terminal you started Snort. Always watch alert_fast.txt in a second terminal with: sudo tail -f /var/log/snort/alert_fast.txt

PART 6 — TESTING ALL FIVE RULES

■ PayliteNG Context

Generate the exact same attack traffic @gr4yh4t sent to PayliteNG and watch Snort 3.0 detect it in real time.

Setup — Open THREE terminals before testing:

SETUP

```
# Terminal 1 — Run Snort 3.0 sudo snort -c /etc/snort/snort.lua -i eth0 -l /var/log/snort -A
alert_fast # Terminal 2 — Watch alerts live sudo tail -f /var/log/snort/alert_fast.txt # Terminal
3 — Generate test traffic (commands below)
```

TEST 1 — ICMP — Verify Snort is Working

Detected by: Snort

BASH — TERMINAL 3

```
# Terminal 3 ping -c 3 localhost
```

✓ Rule 1 — ICMP detected

```
# In Terminal 2 (alert_fast.txt): 01/20-09:00:01 [**] [1:1000001:1] "TECHRISE ICMP Detected" [**]
[Priority: 0] {ICMP} 127.0.0.1 -> 127.0.0.1 ← Rule 1 fires 3 times — once per ping ← Snort 3.0 working
correctly
```

Note: In Snort 3.0 alert_fast format the message appears in quotes — different from Snort 2.x console format.

TEST 2 — SSH Brute Force — detection_filter working

Detected by: Snort

BASH — TERMINAL 3

```
# Terminal 3 — generate 8 failed SSH attempts for i in {1..8}; do ssh -o StrictHostKeyChecking=no \
-o ConnectTimeout=1 \ wronguser@localhost 2>/dev/null sleep 0.5 done
```

✓ Rule 2 — SSH brute force detected

```
01/20-09:00:05 [**] [1:1000002:1] "PAYLITENG SSH Brute Force Detected" [**] [Priority: 0] {TCP}
127.0.0.1:xxxxx -> 127.0.0.1:22 ← Alert fires after the 5th attempt (detection_filter threshold) ← Would
have caught @gr4yh4t at 03:47:20 on January 13th
```

TEST 3 — Anonymous FTP Login

Detected by: Snort

BASH — TERMINAL 3

```
# Terminal 3 ftp -n localhost << 'EOF' quote USER anonymous quit EOF # If ftp not installed: sudo apt install ftp -y
```

✓ Rule 3 — Anonymous FTP detected

```
01/20-09:00:10 [**] [1:1000003:1] "PAYLITENG Anonymous FTP Login Attempt" [**] [Priority: 0] {TCP} 127.0.0.1:xxxxx -> 127.0.0.1:21 ← Fires the moment USER anonymous appears in packet ← In PayliteNG the backup was downloaded 3 seconds after this
```

TEST 4 — SQL Injection in HTTP

Detected by: Snort + DVWA

BASH — TERMINAL 3

```
# Confirm DVWA running docker ps | grep dvwa # If not: docker start dvwa # Terminal 3 — generate SQL injection traffic curl 'http://localhost/dvwa/vulnerabilities/sqli/?id=1+UNION+SELECT+user,password+FROM+users--' \ -H 'Cookie: security=low; PHPSESSID=abc123' \ -o /dev/null -s # Also test in browser: DVWA → SQL Injection → 1' UNION SELECT user,password FROM users--
```

✓ Rule 4 — SQL injection detected

```
01/20-09:00:15 [**] [1:1000004:1] "PAYLITENG SQL Injection UNION SELECT" [**] [Priority: 0] {TCP} 127.0.0.1:xxxxx -> 127.0.0.1:80 ← http_uri sticky buffer caught UNION SELECT in the URL ← Same attack 203.0.113.88 used against PayliteNG
```

TEST 5 — Large FTP Transfer

Detected by: Snort

BASH — TERMINAL 3

```
# Terminal 3 — simulate large outbound FTP packets dd if=/dev/zero bs=1024 count=20000 2>/dev/null | nc localhost 21 2>/dev/null # If Metasploitable2 is running: # curl -s ftp://192.168.43.16/ --user anonymous: -o /dev/null
```

✓ Rule 5 — Large transfer detected

```
01/20-09:00:20 [**] [1:1000005:1] "PAYLITENG Large FTP Transfer — Possible Exfiltration" [**] [Priority: 0] {TCP} 127.0.0.1:21 -> 127.0.0.1:xxxxx ← dsize:>10000 matches large outbound FTP packets ← A 45MB download generates hundreds of these alerts
```

Test Results — Record Your Findings

Test	Attack	Rule	Result	Snort 3.0 Change Used
1	ping -c 3 localhost	ICMP Detected	■ Fired ■ Failed	None — same as 2.x
2	8 SSH failed logins	SSH Brute Force	■ Fired ■ Failed	detection_filter
3	FTP USER anonymous	Anon FTP Login	■ Fired ■ Failed	None — same as 2.x
4	UNION SELECT in URL	SQL Injection	■ Fired ■ Failed	http_uri sticky buffer
5	Large FTP packets	Large Transfer	■ Fired ■ Failed	None — same as 2.x

PART 7 — SAVE ALERTS AND CONNECT TO SPLUNK

View and Save Snort 3.0 Alert Logs

Snort 2.x (old)	Snort 3.0 (use this)
`# Snort 2.x alert file sudo cat /var/log/snort/alert`	`# Snort 3.0 alert file sudo cat /var/log/snort/alert_fast.txt`

BASH

```
# Count alerts that fired sudo grep -c '\[\*\*\]' /var/log/snort/alert_fast.txt # Show only PayliteNG alerts sudo grep 'PAYLITENG' /var/log/snort/alert_fast.txt # Which rules fired most sudo grep 'PAYLITENG' /var/log/snort/alert_fast.txt \ | awk -F'"' '{print $2}' | sort | uniq -c | sort -rn # Save to lab folder mkdir -p ~/labs/week7 sudo cp /var/log/snort/alert_fast.txt ~/labs/week7/snort_alerts.txt echo 'Saved'
```

Connect Snort 3.0 Alerts to Splunk

Feed Snort 3.0 Alerts into Your Existing SIEM

- In Splunk: Settings → Add Data → Monitor → Files and Directories
- File: /var/log/snort/alert_fast.txt (Snort 3.0 file — not alert)
- Source type: snort (or _json if using -A alert_json)
- Index: payliteng

SPLUNK SEARCH

```
# Unified search — all three detection layers together index=payliteng OR index=dvwa OR index=wazuh | table _time source _raw | sort -_time # This shows the complete PayliteNG attack chain in one view: # SSH brute force from Snort # SQL injection from Snort + DVWA Apache log # File changes from Wazuh # All sorted by time — the full story
```

■ **For richer Splunk integration, run Snort 3.0 with -A alert_json flag. JSON format gives Splunk structured fields to search on — rule ID, source IP, destination IP — making SPL queries much more precise.**

FINAL CHECKLIST — DAY 1 COMPLETE

✓ All items complete — Snort 3.0 fully operational

- ■ Snort --version confirms Version 3.x (not 2.x)
- ■ /etc/snort/snort.lua exists (not snort.conf)
- ■ /etc/snort/rules/local.rules created and included in snort.lua
- ■ sudo snort -c /etc/snort/snort.lua --warn-all passes
- ■ Rule 1: ICMP (sid:1000001) — identical in 3.0
- ■ Rule 2: SSH brute force using detection_filter (not threshold)
- ■ Rule 3: Anonymous FTP content match (identical in 3.0)
- ■ Rule 4: SQL injection using http_uri sticky buffer with comma syntax
- ■ Rule 5: Large FTP dsize (identical in 3.0)
- ■ Snort running with: snort -c snort.lua -i eth0 -A alert_fast
- ■ Alerts watching: sudo tail -f /var/log/snort/alert_fast.txt
- ■ TEST 1: ICMP rule fired
- ■ TEST 2: SSH brute force rule fired (detection_filter working)
- ■ TEST 3: Anonymous FTP rule fired
- ■ TEST 4: SQL injection rule fired (http_uri sticky buffer working)
- ■ TEST 5: Large FTP transfer rule fired
- ■ /var/log/snort/alert_fast.txt saved to ~/labs/week7/snort_alerts.txt
- ■ Snort alerts ingested into Splunk from alert_fast.txt